TICTACTOANG

Group Project Report

Nguyen Q. Khanh, Tran H. Minh, Hoang M. Thang,
Nguyen D. G. Phat, Truong K. Minh
Royal Melbourne Institute of Technology Vietnam
School of Science, Engineering, and Technology
Lecturer: **Dr. Tri Huynh**

23th May 2026

Table of contents

A. List of Figures

B. List of Tables

1. Introduction

TicTacToang is a full-stack online Gomoku platform built as the capstone deliverable for COSC2769/COSC2808. It implements three gameplay modes - single-player AI, local two-player, and real-time online multiplayer via WebSocket within a Modular Monolith backend (Node.js/Express/MongoDB) and a React 19 single-page frontend.

The system serves **two authenticated roles** and one **unauthenticated access class:**

Table 1: System Target User Classes and Authorization

Role	Auth Required	Core Capabilities	Gated Features
Guest	No	View landing page	All gameplay, profile, admin
PLAYER	Yes (JWT cookie)	Play all game modes, profile, history, match replay	Replay, premium chat (Premium only)
ADMIN	Yes (JWT cookie, ADMIN role)	Dashboard metrics, player management, room monitoring, force-close	Player-facing gameplay

The scope of the project covers: stateless JWT-based authentication with brute-force account lockout; profile management with Cloudinary-hosted avatar images; a three-tier AI engine **(Easy/Medium/Hard)**; real-time online game rooms with 60-second disconnect resilience; move-by-move match replay behind a PayPal-integrated Premium subscription; and an administrative portal with live room monitoring and EventBus-driven cascade disconnection.

This report documents the architectural decisions, implementation details, and technology trade-offs of TicTacToang. Where non-trivial choices were made, alternatives are presented and justified with reference to published sources.

2. Project Description

2.1 Main Features and User-Facing Functions

Authentication and Registration

Registration collects a username, email, password, confirmPassword, and country. Password strength is enforced by four concurrent rules, minimum 8 characters, one uppercase letter, one digit, one special character, validated on both the client (useFormValidation) and the server (auth.validator.js). Failed login attempts trigger a brute-force lockout after five attempts within 60 seconds, with a per-second countdown returned in the error response.

Game Modes and Customisation

Three modes are available from GameModeSelect: Single Player (against an AI bot at Easy, Medium, or Hard difficulty), Local Two-Player (two humans sharing a device), and Online Multiplayer (real-time via Socket.IO). Before each match, players configure board size (10×10 or 15×15), grid style (Classic / Neon / Block), and one of six marker variants.

Gameplay Engine and AI

The win condition is five consecutive marks on any axis. On win, the server-side checkGomokuWin function (src/utils/gomoku.util.js) returns the exact five-cell coordinate array in algebraic notation (e.g., A1–E1), which drives the WinOverlay animation. Three AI strategy modules handle single-player mode: Easy uses random adjacency, Medium applies a pure defensive heuristic scan, and Hard combines heuristic evaluation with Minimax and alpha-beta pruning (depth bounded by a 1,200 ms time limit).

Real-Time Online Multiplayer

Online rooms follow the lifecycle WAITING → READY → PLAYING → CLOSED or ABORTED. A 60-second disconnect grace period prevents match abandonment on transient network drops: the server holds the room open and emits a countdown to the remaining player, then rehydrates the board state on reconnect.

Profile, Premium Subscription, and Admin Portal

The Profile page consolidates contact information, avatar (Cloudinary CDN), match history with filters, and subscription status. Premium access is unlocked via a PayPal REST v2 payment and grants match replay and online chat features. The Admin Portal provides a real-time dashboard, player management with cascade socket disconnection via EventBus, and force-close of active rooms.

2.2 Use Cases Supported

UC-1: Online Multiplayer Match Lifecycle

Trigger: Two geographically separated players initiate a match via the live arena.

Execution:

1. Player A creates a new room, setting its initial state to WAITING.
2. Player B discovers the room in the live arena list and joins, transitioning the state to READY.
3. Both players confirm their readiness, prompting the server to shift the room to PLAYING and broadcast the start signal.
4. Moves are transmitted asynchronously via WebSockets, instantly synchronising the board state across both clients without HTTP polling.

Outcome: The server-side algorithm detects a five-in-a-row win condition, terminates the active room, saves the session payload, and immediately offers a rematch option.

UC-2: Disconnect and State Rehydration

Trigger: A player experiences a sudden network drop or browser closure during a live online match.

Execution:

1. The server detects the severed socket connection and immediately triggers a 60-second suspension timer.
2. The opponent receives a notification, and their interface displays a countdown overlay.
3. The disconnected player navigates back to the application before the timer expires.
4. The frontend calls the authentication endpoint, which detects the unfinished match and issues an automatic rejoin command.

Outcome: The server cancels the abort timer and pushes the exact current board state back to the returning player (state rehydration), allowing the match to resume without any constraints.

UC-: Admin Account Deactivation with Cascade Ejection

Trigger: An administrator suspends a user account due to platform policy violations.

Execution:

1. The admin issues a PATCH request to the user management endpoint.
2. The HTTP layer updates the authentication module, setting the target account status to inactive.
3. To maintain strict decoupling, the system emits an internal event via the EventBus rather than invoking the WebSocket directly.
4. The Socket.IO namespace handler listens to this event and identifies any active rooms containing the target user.

Outcome: The system forces an immediate room closure, gracefully notifies the opponent, and severs the target user's socket connection instantly to enforce the ban across all layers

2.3 Design Constraints and Specification Alignment

System Architecture & Deployment

- Backend Design: Uses a Modular Monolith with an N-Tier structure.

- Data & Communication: Modules communicate through exposed interfaces with DTO-based response sanitization.
- Frontend Structure: Uses a centralized REST helper and separated UI, hooks, and services.
- Deployment: Fully cloud-deployed for global access.

Security & Access Control

- Authentication: Uses JWT in HttpOnly cookies with PLAYER and ADMIN roles.
- Data Protection: Applies input validation and secure password hashing.
- Access Management: Uses rate limiting and role-based middleware for endpoint protection.

Game Engine & Real-Time Sync

- WebSocket Integration: Real-time multiplayer and chat rely on server-controlled sockets.
- Game Rules: Supports fixed 10×10 and 15×15 boards with a 5-mark win condition.
- AI Mechanics: Includes Random, Defensive, and Offensive AI difficulty levels.
- Match Replay: Uses algebraic notation for consistent move history tracking.

Subscriptions & Administration

- Payment Flow: Integrates PayPal for real-world payment processing.
- Admin Authority: Allows admins to manage account states and terminate active rooms remotely.

2.4 Reflections and Lessons Learned

Technical Insights

- An internal EventBus resolved circular dependencies between HTTP services and the WebSocket layer. The pub-sub mechanism allows system layers to communicate without tight structural coupling.
- Standard HTTP polling was too slow for real-time multiplayer. We used WebSockets for live state synchronization and RESTful APIs for data persistence and authentication.
- The Repository Pattern isolated database queries from business services. Database indexing and query optimizations later required no changes to the service logic.

Design Insights:

- We applied the Open-Closed Principle by storing UI theming, game rules, and component variations as data configurations. This separates presentation from core logic, so new game modes do not require changes to rendering components.
- The replay system requires accurate historical data. We embedded user data directly into the match session payload at creation. This prevents data corruption when users change their profile information later.
- We separated concerns to manage real-time transitions. Custom hooks and local state stores handled network lifecycle events and cleanup protocols.

Teamwork Insights:

- Weekly code freeze on Monday, and immediate bug logging to the sprint board for refinement
- Extremely fast teammate communication via Discord, which only takes approximately 30 minutes to reply to any urgent information
- Workflow from Backend APIs had to be documented in **ENDPOINTS.md** before frontend integration to prevent mismatches.

2.5 Requirement Fulfilment Summary

The system implementation covers all functional and non-functional requirements defined in the original SRS. We successfully deployed the dual-layer authentication, real-time multiplayer coordination, and administrative oversight modules using a modular N-Tier architecture. The inclusion of premium content and AI-driven difficulty tiers provides the intended feature set without compromising system stability. By enforcing strict API contracts and a configuration-driven design,

we ensured that the platform remains extensible for future updates. This delivery meets the technical specifications and operational goals established in our project roadmap.

3. Implementation Details

3.1 Architecture Overview

3.1.1 Architecture Diagram

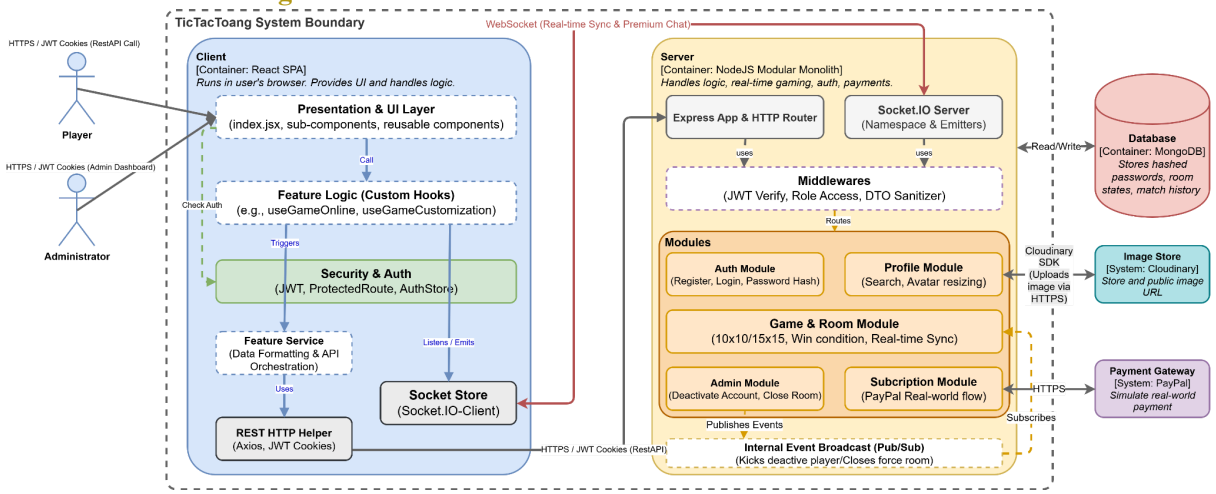

Figure 1: Tictactoang Container Diagram [13]

The Container Diagram shows the high-level architecture of the TicTacToang system and its interaction with users and external services.

- **Client (React SPA):** The frontend container running in the user's browser, serving as the primary interface. It implements a strict internal **N-Tier** flow: The **Presentation & UI Layer** delegates business logic to **Custom Hooks**, which subsequently triggers the **Feature Service** layer for data formatting and API orchestration. Security is centrally managed by a **Security & Auth layer**. All outbound communication is routed through two dedicated gateways: the **REST HTTP Helper** for synchronous HTTPS requests and the **Socket Store** for establishing a persistent WebSocket connection for real-time synchronization.
- **Server (Node.js Modular Monolith):** The backend container responsible for core **business logic**, **real-time game state management**, and **security validation**. Inbound traffic via the **Express App** (HTTP) or **Socket.IO Server** must pass through a centralized **Middleware** layer (handling JWT verification, role-based access control, and DTO sanitization) before reaching the business logic. The logic is strictly organized into cohesive bounded contexts (Modules): **Auth**, **Profile**, **Game & Room**, **Admin**, and **Subscription**. To ensure loose coupling, these modules communicate asynchronously via an **Internal Event Broadcast** (Pub/Sub) pattern (e.g., the Admin module publishing a user deactivation event that the Game module subscribes to for kicking the player).
- **External Systems:** The backend securely integrates with three **external platforms** to offload specific responsibilities. **MongoDB** acts as the primary Database for storing encrypted user credentials, real-time room states, and match history. The **PayPal gateway** is integrated via HTTPS to simulate secure, real-world payment processing. Lastly, the Profile module utilizes the **Cloudinary SDK** to offload the storage, delivery, and optimization of static image assets (user avatars).

3.1.2 Component Diagrams - Backend

To see the backend component diagram, please refer to this link: Tictactoang-BE-Diagram

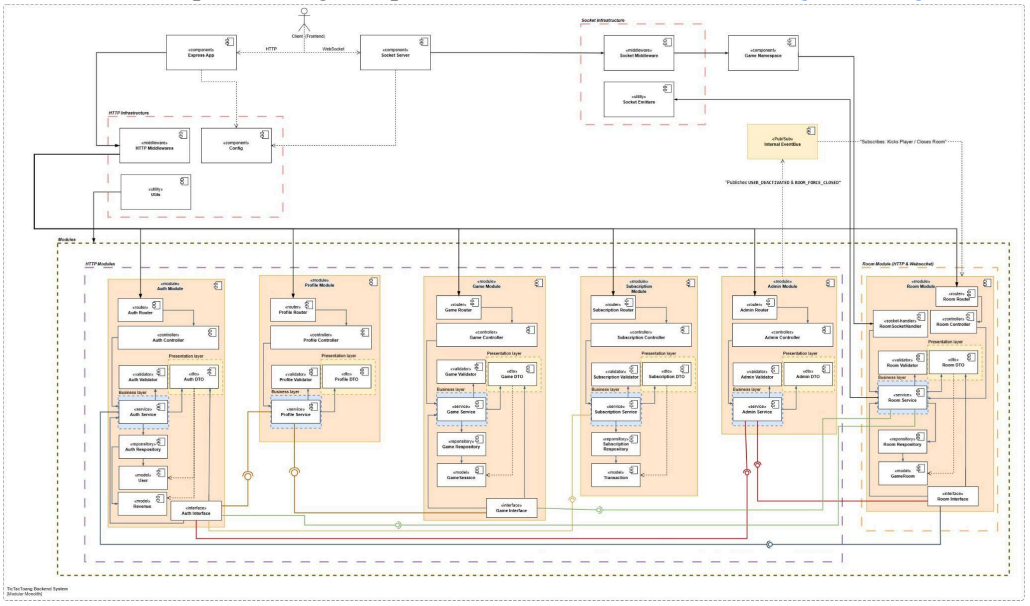

Figure 2: Tictactoang Backend Component Diagram [13]

Backend follows a **Modular Monolith** built on a strict **N-Tier** architecture, dividing its backend into isolated bounded contexts (Auth, Room, Game, Subscription, Profile, Admin) that independently manage their own data and business logic as shown in **Figure 2**. It employs a **Socket-first** strategy, using standard HTTP REST APIs for transactional operations like authentication, profiles, and payments, while reserving WebSockets exclusively for real-time multiplayer gameplay, chat, and state synchronization. To maintain strict domain boundaries, modules never directly access each other's models; instead, they interact synchronously via defined **Interfaces** and asynchronously through an internal **EventBus (Pub/Sub)**, ensuring fully decoupled execution across the entire platform.

The exact folder tree from the ***"server"*** codebase is structured as follows to enforce this architecture:

```
src/
├── app.js                  # Express app initialization and route mounting
├── index.js                # Server entry point, DB connection, and Socket.io
├── config/                 # Environment-dependent configurations
├── middlewares/            # Global request-level middleware chain
├── modules/                # Domain-driven modular monolith (6 modules)
│   ├── admin/              # Admin dashboard and player management
│   ├── auth/               # User authentication and session management
│   ├── game/               # Match history and replay persistence
│   ├── profile/            # User profile updates and avatar management
│   ├── room/               # Live room orchestration and real-time sync
│   └── subscription/       # Payment transactions and premium logic
├── sockets/                # Real-time multiplayer communication handlers
├── seed/                   # Database seeding for development and testing
├── tests/                  # Automated test suites
└── utils/                  # Shared utility functions across modules
```

Note: The internal structure of the admin, game, room, profile, and subscription modules follows the identical architectural pattern demonstrated in the auth module (controllers, services, repositories, models, routes, dtos, validators, interfaces).

N-Tier Flow Direction and Enforcement: The flow of data in the N-tier structure is strictly a one-way process: Route → Controller → Service → Repository → Model. In order to maintain the integrity of the Modular Monolith architecture, the architectural documentation requires that: Routes and socket handlers should not communicate directly with models; services should not communicate with repositories directly; and cross-module communications should happen via an interface and not by importing services from another module.

Justification for the Repository Layer: Repository tier is a central location for all query logic that accesses the database. It facilitates easy implementation of future optimizations in queries (such as indexes or changing the aggregate pipeline). The repository design is in direct compliance with the Software Requirements Specification (SRS) requirement A.1.2 and also follows the Data Access Object pattern described in [2].

3.1.3 Component Diagrams - Frontend

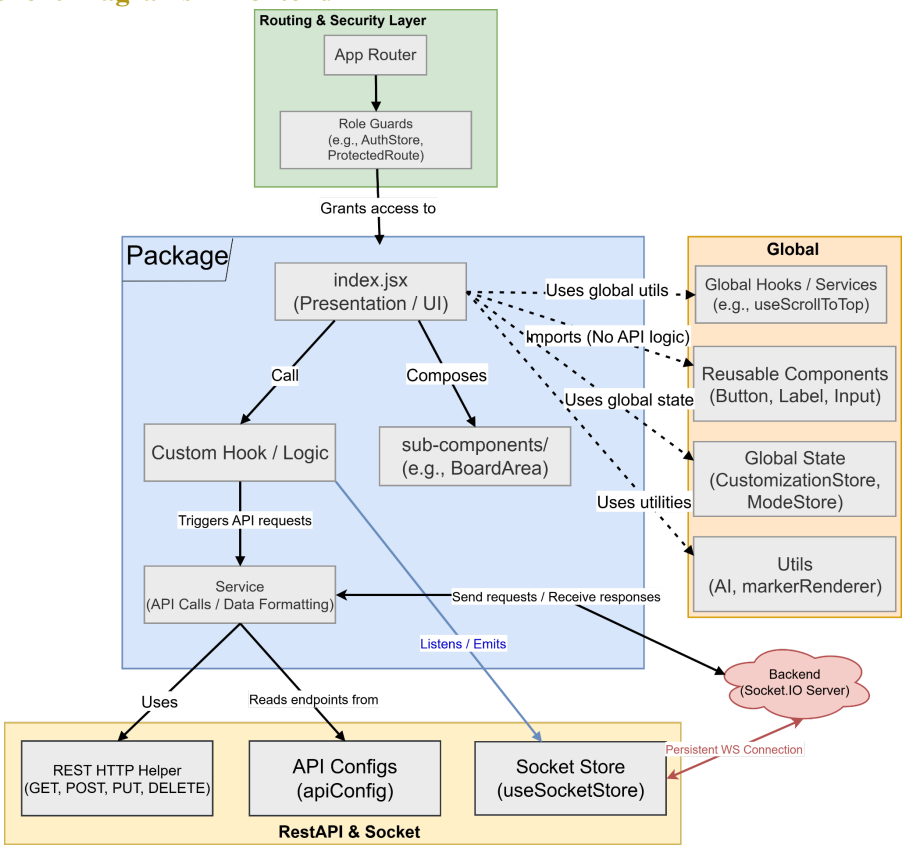

Figure 3: Frontend Architecture Diagram [13]

The diagram illustrates the modular and layered architecture of the TicTacToang frontend, adhering to the N-Tier design pattern (SRS A.1.3, A.3.a). The architecture is structured into four primary scopes:

1. **Routing & Security Layer:** The entry point of the application, utilizing the **App Router** to manage navigation. Access is strictly mediated by **Role Guards** (e.g., AuthStore, ProtectedRoute), ensuring role-based authorization before rendering any UI elements.
2. **Feature Layer (Package):** Encapsulates the core logic and presentation of individual features. Following a decoupled component design (SRS A.3.b), the ***index.jsx*** acts as the primary UI presentation, composing **sub-components** (e.g., BoardArea) and delegating logic to a **Custom Hook**. The hook, in turn, triggers the **Service layer** to format data and orchestrate API requests, maintaining a strict separation of concerns.

3. **Global Scope:** Contains universally accessible utilities and states. The Presentation layer ***index.jsx*** dynamically imports **Reusable Components** (e.g., NavigationBar, Footer), relies on **Global State** (e.g., CustomizationStore) for consistent cross-page data, and utilizes **Global Hooks** and **Utils** (e.g., AI logic, Marker Renderers) to enforce dry (Don't Repeat Yourself) code practices.
4. **RestAPI & Socket Layer:** Serves as the communication bridge. The Feature Service exclusively utilizes a centralized **REST HTTP Helper** configured by **apiConfig** to execute standard CRUD operations. Concurrently, the Custom Hook maintains bidirectional communication with the Backend via the **Socket Store**, emitting and listening to real-time events through a persistent WebSocket connection.

This architectural approach ensures high maintainability, strict security boundaries, and seamless real-time state synchronization across the platform.

3.1.4 Entity-Relationship Diagram

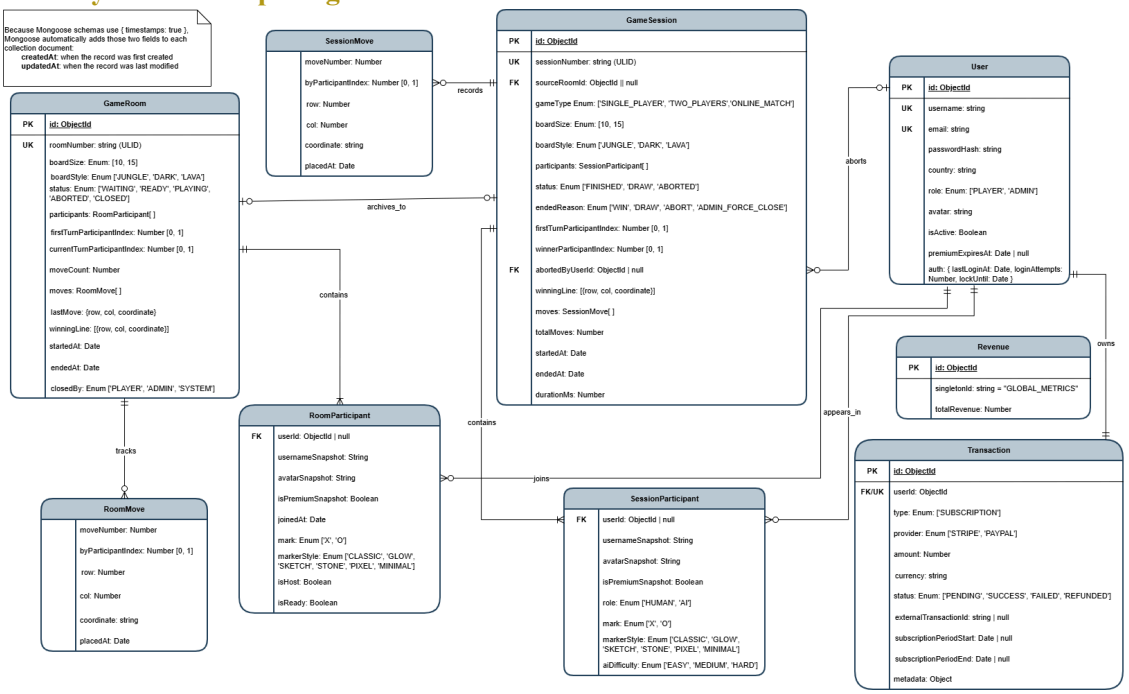

Figure 4: Tictactoang ERD [13]

The TicTacToang database completely separates live game states (GameRoom) from immutable historical records (GameSession) to prioritize query performance and automated cleanup over strict normalization.

Here are the key non-obvious design decisions:

- **Historical Snapshotting:** Profile data (usernames, avatars, premium status) is embedded as static snapshots within game records. This denormalization ensures past match replays remain historically accurate even if a user later updates their profile.
- **Embedded Replay Payloads:** Match moves are stored as arrays of embedded sub-documents (with _id fields disabled) directly inside the session. This allows the app to fetch an entire match and its complete replay sequence in a single, highly efficient database read.
- **Anti-Fragmentation ULIDs:** Shareable reference codes (roomNumber and sessionNumber) are generated using ULIDs. This prevents MongoDB B-Tree index fragmentation by maintaining chronological sortability.

- **Aggressive TTL Automation:** Native MongoDB Time-To-Live (TTL) indexes automatically manage data bloat by self-destructing GameRoom documents shortly after a match ends or is abandoned. Transaction documents are also automatically deleted the exact moment a subscription ends.
- **Singleton Metric Aggregation:** The Revenue model operates as a forced singleton using a fixed singletonId of "GLOBAL_METRICS". This maintains running financial tallies without requiring expensive aggregation queries across historical data.
- **Destructive 1-to-1 Transactions:** The Transaction schema enforces a strict unique constraint on userId. Instead of maintaining a ledger of past payments, a new subscription purchase intentionally overwrites the user's previous transaction record.

3.1.5 Sequence Diagrams for Three Complex Flows

Note: Please click here to view more HD and zoomable diagrams Group1_Sequence_Diagrams

This section presents three sequence diagrams showing interactions between the React client, local hooks, WebSocket handlers, and N-Tier backend services.

3.1.5.1 Real-time Online Match Flow

This diagram represents the event-based bi-directional interaction of the user with Socket.io while playing a multiplayer game. It shows the synchronization of the active rooms, the validation of transactions, and the cross-module communication in order to save results after the game.

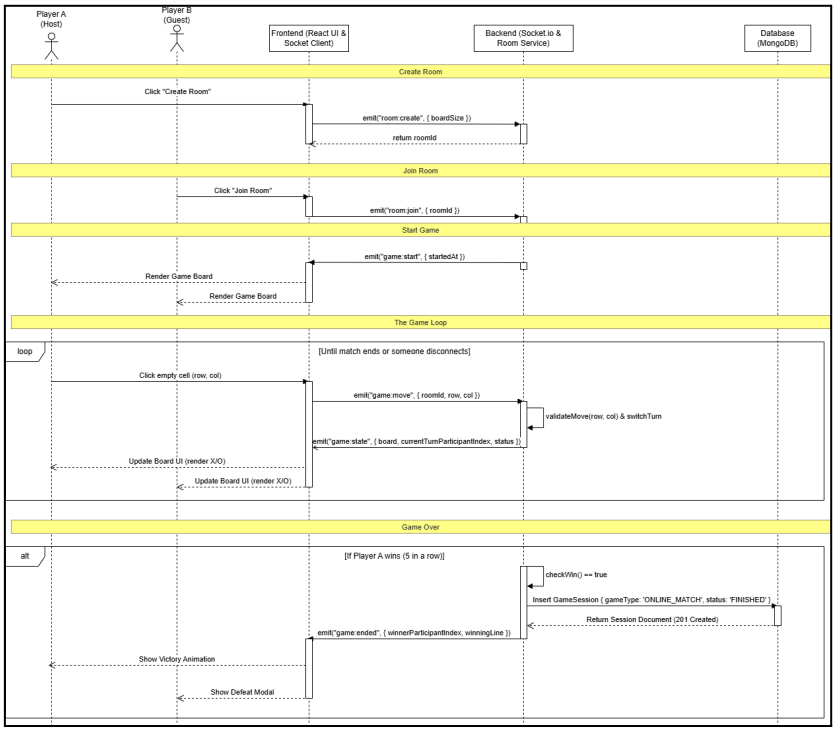

Figure 5: Overview of Real-time Online Match Flow [13]

3.1.5.2 Play Hard AI Flow

This flow depicts the runtime performance of the multiplayer simulation with the computer on the client side. This represents the process of how the Minimax algorithm with Alpha-Beta pruning has been implemented on the client side of the interface that handles artificial locks and delays during the gameplay session before saving the final record to the database.

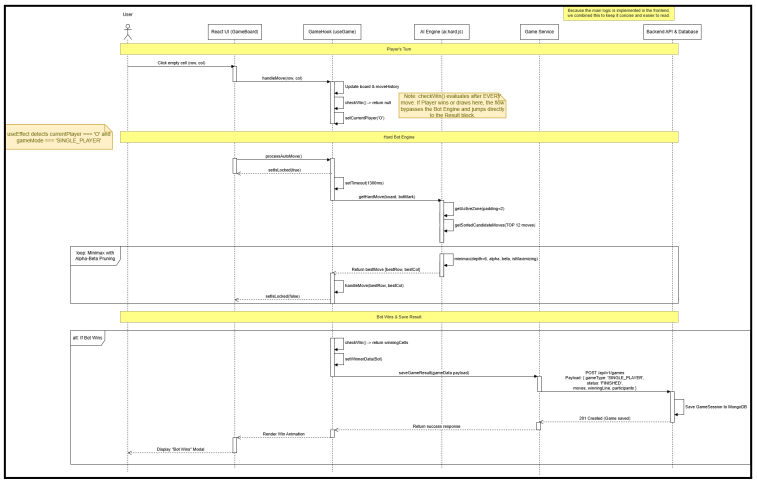

Figure 6: Overview of Play Hard AI Flow [13]

3.1.5.3 Match Replay Execution Flow

The flow diagram presents how the state hydration process is conducted. Specifically, it shows how the historical game information was pulled out only once through a RESTful HTTP GET request and then restored piece-by-piece using React hooks combined with timed intervals.

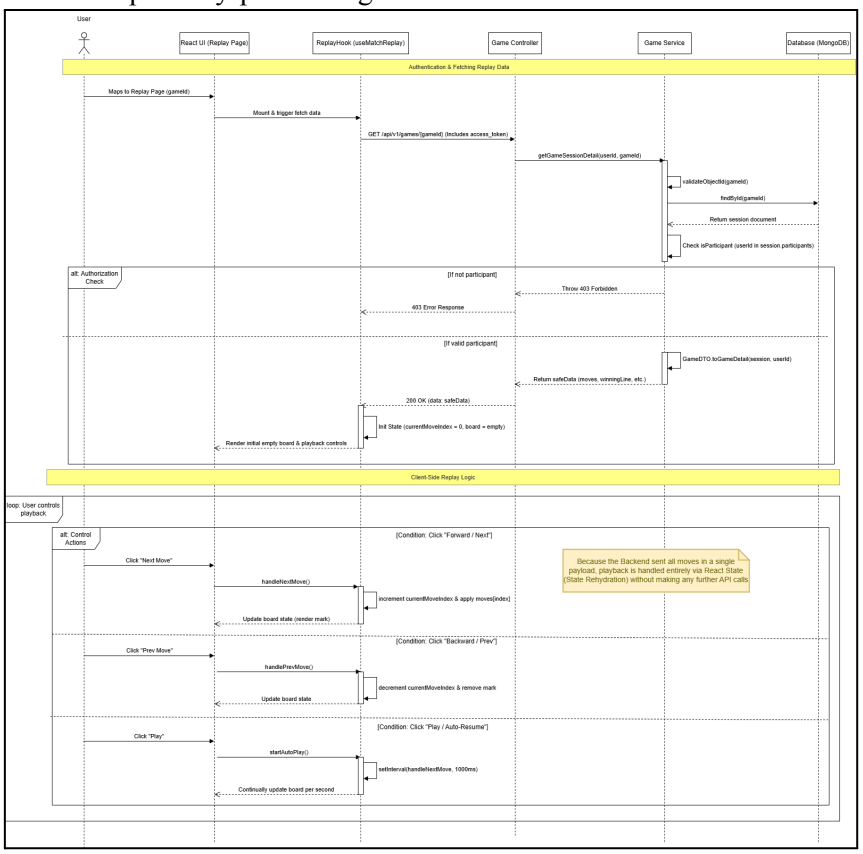

Figure 7: Overview of Match Replay Execution Flow [13]

Note: In order to avoid cluttering the page due to the page budget limit, all the high-quality architecture diagrams have been stored in a shared cloud storage repository asset [13] for easy access

3.2 Frontend and Backend Implementation

3.2.1 Frontend Technologies and Justification

3.2.1.1 File and Folder Organization

```
client/
├── docs/                    # Project documentation
├── public/                  # Public static assets
├── src/                     # Main React application source code
│   ├── assets/              # Internal assets (images, audio, themes)
│   ├── components/          # Reusable UI components
│   ├── config/              # Constants, API, and configurations
│   ├── hooks/               # Custom React hooks
│   ├── pages/               # Main UI pages (organized by roles)
│   ├── routes/              # Routing and route protection
│   ├── services/            # HTTP API request handlers
│   ├── stores/              # Global state management
│   └── utils/               # Utility functions and helpers
├── .eslintrc.js / etc       # System and linter configuration files
├── package.json             # Project metadata and dependencies
└── vite.config.js           # Vite bundler configuration
```

Note: The architecture combines feature-based and role-based organization.

The pages/ directory is divided by user roles (Admin/, Guest/, Player/). Each page functions as an independent module with its own sub-components to manage access control.

State management is separated by domain within the stores/ directory (e.g., AuthStore, ModeStore, SocketStore, ChatStore) rather than relying on a single state file. This handles the data for real-time sockets and themes.

The project isolates UI components, business logic (hooks and stores), and data fetching (services) to clarify data flow for developers.

3.2.1.2 Zustand Store Architecture

Monolithic global state was avoided in favor of six purpose-scoped stores. This follows the principle of minimal shared state: only data requiring persistence across navigation or component coordination resides outside local scope.

- **AuthStore:** Manages session, user identity, and authentication lifecycle using in-memory persistence.
- **CustomizationStore:** Manages board size, grid style, and marker variants using in-memory persistence.
- **ModeStore:** Manages game mode, AI difficulty, and starting player using in-memory persistence.
- **AudioStore:** Manages background music toggle using localStorage persistence.
- **ChatStore:** Manages online messages and unread count using in-memory persistence.
- **Socket Store:** Manages Socket.IO instance and connection state using in-memory persistence.

This approach aligns with the Zustand rationale: small, focused stores prevent over-fetching and minimize cascading re-renders across the component tree [9].

3.2.1.3 HTTP Layer and Interceptors

To standardise client-server communication, we implemented **HttpHelper** as a singleton **Axios wrapper**. It automatically enforces withCredentials: true for secure cookie transmission, strips unnecessary Axios wrapper objects, and handles FormData detection to prevent forcing JSON content-types on multipart uploads.

Crucially, it utilizes an interceptor to decouple authentication state from individual API calls by emitting a global DOM event upon detecting a **401 Unauthorized** response.

3.2.2 Backend API routes with purpose, request, and response data structures

The backend provides a comprehensive RESTful API, aimed at dealing with transactions within different bounded contexts. The following table shows some of the key endpoints and their roles, along with the main data models used by them.

Endpoint	Method	Purpose	Request Structure	Response Structure
AUTH Module				
/api/v1/auth/register	POST	Register a new player account	{ username, email, password, country }	201 Created: { user }
/api/v1/auth/login	POST	Authenticate user & set HTTP-only session cookie	{ identifier, password }	200 OK: { user } + Cookie
/api/v1/auth/check-auth	GET	Validate session and bootstrap app state	(None)	200 OK: { user, activeRoom }
PROFILE Module				
/api/v1/profile/overview	GET	Aggregate user stats and recent games	(None)	200 OK: { user, stats, subscription, recentGames[] }
/api/v1/profile/avatar	POST	Upload or replace avatar (Cloudinary via Multer)	{ avatar: File }	200 OK: { avatar_url }
GAME Module				
/api/v1/games	GET	Paginated game history with filters	?page, limit, gameType, sort	200 OK: { items[], total, page, limit }
/api/v1/games	POST	Save completed local/AI game session	{ gameType, participants[], moves[], winner... }	201 Created: { GameSession }
ROOM Module				
/api/v1/rooms	GET	Arena snapshot (Joinable/Active rooms)	?status, boardSize, page	200 OK: { items[], total, page, limit }

SUBSCRIPTION Module				
/api/v1/subscription/create-order	POST	Generate PayPal payment link	(None)	201 Created: { orderId, links[] }
/api/v1/subscription/capture-order	POST	Validate PayPal payment & activate premium	{ orderId }	200 OK: { success, transaction, premiumExpiresAt }
ADMIN Module				
/api/v1/admin/dashboard	GET	Aggregated platform metrics	(None)	200 OK: { totalUsers, activeRooms, totalMatches... }
/api/v1/admin/rooms/:id	DELETE	Force close a room (kicks players via Socket)	Path: {id}	200 OK: { message }

To maintain strict documentation standards and preserve the 25-page report limit, only core transactional endpoints are illustrated above. The full API contract, which covers all CRUD operations, continues to be documented using an interactive OpenAPI/Swagger UI available at our backend deployment [14] and ENDPOINTS.md.

WebSocket Contract: While REST API's process transactions, real-time synchronization for multi-player games is done solely using WebSockets. The following table describes the basic two-way event emitters:

Table 2: Websocket Contract

Event Name	Direction	Key Payload	Description
room:create	Client → Server	{ boardSize, markerStyle }	Host creates a new multiplayer lobby
game:move	Client → Server	{ roomId, row, col }	Player submits a board coordinate during their turn
room:updated	Server → Client	{ room: { participants... } }	Broadcasts lobby state changes to all joined players
game:state	Server → Client	{ board, turn, status }	Broadcasts the updated board after a valid move
game:ended	Server → Client	{ winnerIndex,	Match finishes; triggers

		winningLine }	client-side victory animations
room:join / room:leave	Client → Server	{ roomId }	Player joins or leaves an existing lobby
room:ready	Client → Server	{ roomId }	Player toggles their ready status
chat:send	Client → Server	{ roomId, message }	Dispatch in-game text message to opponent
chat:message	Server → Client	{ sender, message, timestamp }	Broadcasts incoming chat message to the room
player:disconnected	Server → Client	{ timeLeft }	Opponent offline; triggers 60s reconnection grace period

3.2.3 Implementation Notes

Modular Monolith and Interface Isolation

The system utilizes a modular monolith architecture, enforcing strict separation of concerns across Controllers, Services, and Repositories. Modules communicate exclusively through defined interfaces instead of accessing internal repositories directly. For example, the Room module archives matches via **GameInterface.createOnlineGameSessionFromRoom(),** and the Admin module aggregates statistics via **GameInterface.getTotalPlatformMatches()**. This design centralizes business logic and prevents cascading failures across module boundaries.

Gomoku Winning Algorithm

Core game validation resides in **src/utils/gomoku.util.js**. The checkGomokuWin function scans the grid along four axes (horizontal, vertical, and both diagonals) starting from the most recent move. It calculates contiguous chain lengths and returns algebraic coordinates (e.g., A1 to E5) upon detecting a five-tile sequence. This server-side validation acts as the single source of truth to prevent client-side manipulation.

Security Architecture and Global Error Handling

The Express middleware pipeline (app.js) executes in a strict order**: CORS (URL-restricted)**, **CookieParser (HTTP-only tokens)**, **BodyParser**, and a global RateLimiter. Routes enforce security using verifyToken (JWT extraction) and authorizeMiddleware (RBAC). Finally, errorMiddleware intercepts exceptions, sanitizes stack traces for production safety, and returns a standard JSON structure ({error, message, cause, valid_example}) to deliver actionable client feedback.

3.3 UI/UX Design

3.3.1 Layout and Responsiveness

The TicTacToang frontend employs a **multi-stage responsive layout system** implemented through conditional rendering in **Layout.jsx** (../src/Layout.jsx), which serves as the **application shell**. Tailwind CSS 4 utility classes provide **responsive breakpoints**. This architecture distinguishes between **three layout contexts**, each optimized for specific page types and viewports constraints:

- **Immersive Routes (‘/game/', ‘/room/online/’):** Full-screen game board with maximum viewport utilization. Completely hidden navigation bar and footer.
- **Constrained Routes (‘/subscription’, ‘/play’, ‘/success’, ‘/cancel’):** Pages requiring fixed viewport height with scrollable content.

- **Default Routes (other routes):** Standard pages with scrollable content.

Responsive Mobile UI: The frontend includes a dedicated **mobile UI layer** for the main game and lobby flows. Instead of simply shrinking desktop layouts, the interface changes structure to keep the primary actions visible and usable on narrow screens.

- Navigation: The top nav collapses into a hamburger menu on small screens so auth actions, music toggle, and page links stay reachable.
- Game Board Layout: Local and online match boards switch from a horizontal arrangement to a vertical stack on mobile, placing player panels above and below the board.
- Player Panels: Player cards compress on small screens so avatars, marker previews, and labels stay visible without horizontal clipping.
- Replay Controls: The replay playbar expands to full width on mobile and the control groups wrap so playback actions remain reachable.
- Room Cards: Lobby cards and marker selector cards expand to full width on mobile, preventing overflow and preserving tap targets.

For visual demonstration of mobile UI, visit **Appendix 7.1: Desktop & Mobile UI**

Page Hierarchy: The page hierarchy across Guest, Player, and Admin sections is as followed:

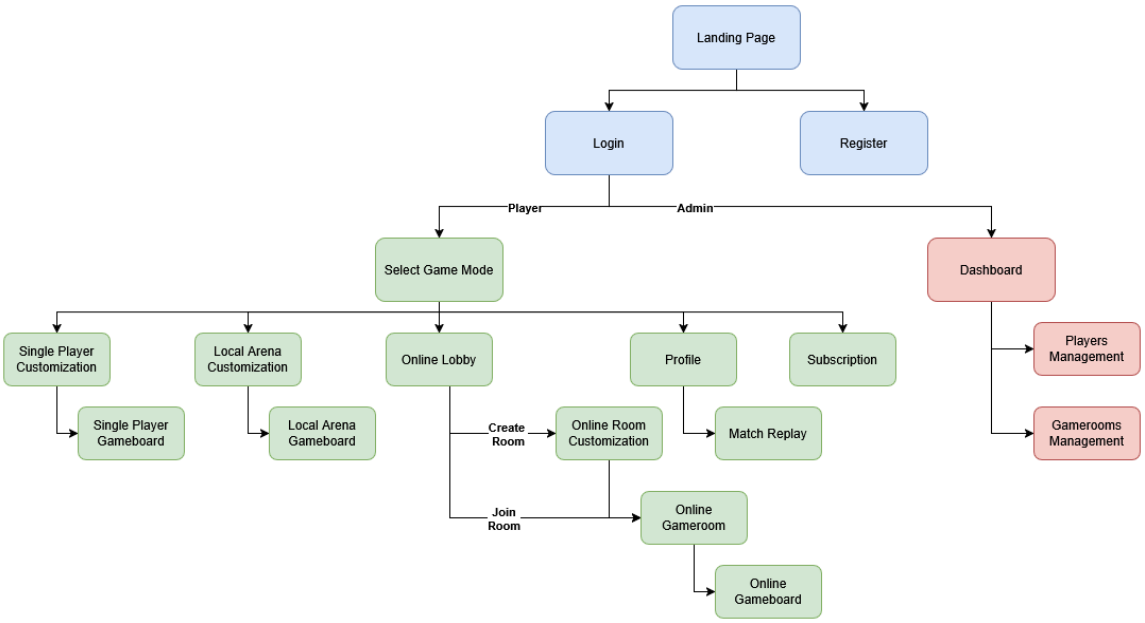

Figure 8: TicTacToang Site Map Diagram

3.3.2 Theming System

The application's visual identity relies on a strict, token-based design system to enforce a consistent retro-arcade aesthetic. We achieved this through centralized configuration and reusable UI components.

Design System Configuration (tailwind.config.js):

- **Semantic Tokens:** We replaced hardcoded hex values with Material Design 3 style semantic tokens (e.g., primary-cyan, surface-container) to ensure consistent color application across all components.
- **Typography Pairing:** The UI exclusively utilizes two Google Fonts: ***Press Start 2P*** for pixel-art headings and labels, and ***IBM Plex Mono*** for highly readable body text.

- **Strict Geometry:** We explicitly overrode all borderRadius utility classes to 0px. Enforcing sharp edges universally distinguishes the game from standard rounded corporate web applications.

Configuration-Driven Theming:

- Game board aesthetics are managed centrally via **src/config/gameThemes.config.js.** Themes are defined as pure data objects. Components simply read this configuration without embedding complex conditional logic, making the system highly extensible.

Two highly reusable components enforce the thematic identity system-wide:

- **ScanLines.jsx:** Renders a fixed CRT monitor scanline pattern. This utilizes a single CSS pseudo-element, adding significant visual depth with zero JavaScript performance cost.
- **CustomMarkers.jsx:** Renders the X and O game pieces. Depending on the user's selected variant, it dynamically outputs either stylized text elements (using the *Press Start 2P* font) or pure inline SVGs for distinct geometric layouts.

3.3.3 Game Board Accessibility

- Keyboard operability: Native <button> cells support keyboard and screen readers.
- Disabled state: Occupied or locked cells cannot receive focus.
- Coordinate labels: Edge labels improve board navigation.
- Active turn indication: Active player uses visual and text indicators.
- Win announcement: Modal clearly displays game results.
- Audio feedback: Sound effects signal moves and outcomes.

3.3.4 Chat Overlay Design

Two components share a common design contract:

Property	ChatOverlay.jsx (Offline)	ChatBox.jsx (Online)
Data source	useChatManager hook - local state + bot taunt scheduler	Socket.IO chat:send / chat:message events
Placement	Fixed bottom-left, z-50, below WinOverlay	Inline panel within GameOnline layout
Message scope	Human vs. bot	Two remote players

Message differentiation by sender:

Sender	Background	Border
Current player	rgba(123,97,255,0.22) - purple	rgba(123,97,255,0.4)
Opponent	rgba(76,201,240,0.10) - cyan	rgba(76,201,240,0.25)
AI bot	rgba(255,61,0,0.15) - orange-red	rgba(255,61,0,0.3)

Unread badge: Toggle button border and text shift from cyan to orange-red when unreadCount > 0 and the panel is closed.

Bot Taunt Scheduler (useChatManager): First taunt fires at 5 seconds (masking the 1,300ms AI thinking delay); subsequent taunts every 18 seconds from a pool of 8 pre-written lines. Active only during SINGLE_PLAYER mode while gameOver === false.

Typing indicator: Three CSS-animated dots (--bounce-delay: 0s / 0.2s / 0.4s) render when the other player is typing, driven by the input onChange handler. Typing indicators are documented as effective social presence signals in CSCW research, reducing perceived wait time and increasing engagement in text-based communication interfaces [16].

3.4 Tools and Package Selection

This section outlines the core technologies and cloud services selected for the TicTacToang platform, justified by architectural constraints and Software Requirements Specification (SRS)

3.4.1 Frontend Ecosystem

Table 3: Frontend Architecture Trade-offs and Tooling Selection

Concern	Selected	Rejected	Reason
UI Framework	React 19	Vue 3, Angular	Unidirectional data flow fits N-Tier separation. Concurrent rendering handles real-time states [6].
Build Tooling	Vite 8	CRA, Next.js	CRA is deprecated. Next.js adds unneeded SSR complexity. Vite offers native ESM and fast HMR [7].
Global State	Zustand 5	Redux Toolkit	Zustand needs minimal boilerplate, avoiding the setup overhead of RTK.
Routing & RBAC	React Router v7	TanStack Router	ProtectedRoute.jsx checks roles against AuthStore. React.lazy reduces bundle size.
HTTP Client	Axios 1.13	Native fetch	Interceptors provide consistent errors and enable global withCredentials for HttpOnly cookies.
Real-time	Socket.IO Client 4.8	Raw WebSocket	Provides built-in event routing, auto-reconnection, and HTTP long-polling fallbacks [8].
Animation	Framer Motion 12	CSS Keyframes	AnimatePresence handles exit animations on removed DOM nodes, unlike pure CSS.
Data Visualisation	Recharts 3	Chart.js	Declarative components avoid the imperative chart.update() required by Chart.js.

3.4.2 Backend, Database, & Real-time

Table 4: Backend Architecture Trade-offs and Tooling Selection

Concern	Selected	Rejected	Reason
Runtime & Framework	Node.js & Express.js	Django, Spring Boot	Event-driven runtime handles concurrent sockets. Express enables N-Tier routing and logic separation.

Database & Validation	MongoDB & Mongoose	PostgreSQL, MySQL	Embeds variable-length game data directly, avoiding relational joins. Mongoose enforces schema validation.
Real-time	Socket.io	Raw WebSockets	Ensures low-latency move replication for network play.
Identifiers	ULID	UUIDv4	Generates chronologically sortable IDs to optimize database indexing.

3.4.3 Security, Media & External Services

- Authentication Stack (JWT, Bcrypt, Express Rate Limit): JWT provides stateless, cryptographic session authorization. Bcrypt hashes passwords to prevent plaintext exposure. Rate limiting acts as a defensive layer against brute-force credential stuffing.
- Image Pipeline (Multer, Sharp, Cloudinary): Multer intercepts file uploads, while Sharp performs high-performance server-side avatar resizing. Processed images are offloaded to Cloudinary, an optimized cloud CDN, fulfilling the profile picture constraint while preserving server bandwidth.
- PayPal REST API & Nodemailer: The PayPal integration utilizes signature-verified webhooks to securely process premium subscriptions. Nodemailer interfaces with SMTP servers to trigger automated transactional emails (e.g., payment confirmations).
- Swagger (OpenAPI): Auto-generates interactive API documentation from backend JSDoc annotations, establishing a clear contract that minimizes integration friction between frontend and backend workflows.

3.4.4 Deployment Infrastructure

- Vercel & Render: The frontend is deployed to Vercel's Edge Network for optimized static asset delivery. The Express backend is hosted on Render, offering robust support for persistent WebSocket connections.
- MongoDB Atlas: A fully managed cloud cluster with strict whitelist configurations to guarantee secure, high-availability data transactions.

3.5 Development Process

3.5.1 Methodology

The team adopted a **SCRUM**-inspired iterative workflow with one-week sprints. The **GitHub Projects board** was configured with the following swim lanes: **Product Backlog**, **Sprint Backlog**, **In-Progress**, **Review**, **Staging**, and **Done**, plus some **Documentation** and **Meeting Minutes** lanes. Every task card includes an assignee, a due date, and a complexity estimate in **priority** (e.g., P0, P1) and **size** (e.g., S, M, L, XL). Hours were not used as an estimation unit, in alignment with SCRUM best practices [9]. In addition, we use **Discord** for communication and to organize meetings. The lecturer was added as a viewer on the GitHub project board from Week 2, satisfying the SRS methodology requirement.

3.5.2 Branch Strategy

The repository uses **a main (protected) branch + feature-branch** strategy. Feature branches are named ***frontend/<module>*** for frontend updates and ***backend/<module>*** for backend code changes (e.g., backend/implement-websocket, frontend/lobby_avatars). Pull requests require at least one peer review before merging. Each sprint release is merged into **TicTacToang-v*X*** to review the whole system before merging to main and continuing to develop.

3.5.3 Sprint Overview

Summarise the sprints briefly:

- **Sprint 1** (Week 5): Authentication and Middleware & Authorization & CRUD Model (Objective: To perfect the highly secure Registration/Login (JWT) feature.)
- **Sprint 2** (Week 6): Core Board & Local Play (Objective: To perfect the advanced TicTacToe board game and the offline 2-player mode.)
- **Sprint 3** (Week 7): AI Opponent - 3 Levels (Objective: Complete the Single Player mode with the Bot.)
- **Sprint 4** (Week 8): Real-time Multiplayer & Chat (Objective: Achieve Ultimo score with real-time online gameplay and chat.)
- **Sprint 5** (Week 9): Profile Management & Game Replay (Objective: A unique feature for managing your personal profile and reviewing game replays.)
- **Sprint 6** (Week 10): Premium Subscription & Admin Portal (Objective: To simulate payment processing and system administration.)
- **Sprint 7** (Week 12): Deployment, Testing & Demo Prep (Objective: Package the project, deploy it to the Cloud, and prepare data for scoring.)

3.6 Visual Demonstration

For detailed demonstration of the website UI, visit: Appendix 8.1

3.7 Feature Checklist

Table 5: SRS-to-Implementation Compliance Matrix

SRS Feature	Description	Status
A.1.1 - A.1.4	N-Tier Layer Architecture (FE + BE)	**Implemented**
A.2.1 - A.2.3	Modular Monolith Backend	**Implemented**
A.2.a - A.2.c	Frontend API Config, HTTP Helper, Role Auth	**Implemented**
A.3.1 - A.3.2	Interface-exposed modules, DTOs	**Implemented**
A.3.a - A.3.b	Package-based Componentization	**Implemented**
A.4.b	Responsive Design for Profile and Admin	**Implemented**
1.1.1 - 1.3.1	Registration with full validation (FE + BE)	**Implemented**
2.1.1 - 2.3.2	Login, brute-force protection, JWT, logout	**Implemented**
3.1.1 - 3.3.1	Profile management, avatar, history filter/sort	**Implemented**
4.1.1 - 4.2.6	All game modes, AI levels, and winner animation	**Implemented**
4.3.1 - 4.3.3	Online multiplayer, chat, match replay	**Implemented**
5.2.1	PayPal subscription, email notification	**Implemented**
6.1.1 - 6.3.3	Admin dashboard, player, and room management	**Implemented**
D.1.1 - D.2.1	Cloud deployment (Render + Vercel-equivalent)	**Implemented**

Extra Features (Beyond SRS Requirements):

1. Swagger API documentation at /api-docs (full OpenAPI 3.0 spec, all endpoints annotated)
2. Retro sound effect for important buttons, winning or losing, and background music
3. Win and Lose customized overlay with animation of the critical moves leading to victory
4. Grace period to wait for the opponent who accidentally disconnects from the game while playing, so they can rejoin the room, and small banner to show their reconnection
5. Background image along with board style
6. Line charts and bar charts in the admin dashboard make it easy to monitor system progress and metrics. (e.g., how many users have registered in 1 week/month)
7. Integrate notifications to users when their account is deactivated and a room is closed by the admin.
8. Detect duplicate logins when someone logs in to the same account from other devices
9. Mobile devices layout are supported
10. Bot taunting chat
11. Email sending feature when registered to premium feature (only on local).

3.8 Known Issues or Limitations

1. Missing Informational Content: The platform lacks static landing pages for user onboarding, including an "About Us" section, system metrics, contact details, and historical context for the game.
2. Limited Premium Scope: The premium subscription tier offers a narrow feature set, currently restricted to match replays and online chat without additional advanced functionalities.
3. Multiplayer Stability at Scale: Intermittent state synchronization errors occur during real-time online matches when subjected to edge-case network conditions or high-load testing.
4. Email sending feature when register to premium does not work when deployed

4. Evaluation

Correctness and expected behavior

We validated API contracts with Postman, testing expected paths, data validation, and role-based access control. Frontend peer reviews caught UI edge cases, race conditions, and invalid form submissions before merging. To verify real-time integration, we ran concurrent browser sessions to test WebSocket room orchestration, move synchronization, and 60-second disconnect recovery. While standard gameplay is stable, stress testing revealed intermittent synchronization errors.

Performance optimization

A MongoDB compound index ({ "participants.userId": 1, createdAt: -1 }) reduced the match history query (GET /api/v1/games) execution time to under 10ms for a dataset of 100 matches per user. Serving avatars through Cloudinary CDN removed media processing bottlenecks from the Express server and improved client page load speeds.

Feedback incorporation

Following instructor feedback, we are prioritizing three specific improvements to the project. We will integrate the PayPal Sandbox API to simulate payment transactions for the premium subscription tier. Additionally, we are refining our Agile management processes by enforcing stricter GitHub project tracking and documenting detailed sprint retrospectives to drive iterative development. Finally, we will update the game board UI to allow players to select their markers independently, removing the current restriction that forces both users to share a pre-defined pair.

Usability, responsiveness, and accessibility

The game board uses semantic HTML <button> elements for keyboard navigation and high-contrast colors for visibility. The UI stacks vertically on mobile viewports. The platform lacks contextual onboarding, such as a landing page, about us page, sending new users directly into the core application to choose game mode.

5. Conclusion

5.1 Strengths of the Application

The TicTacToang platform successfully delivers a highly responsive, real-time multiplayer experience. The key to success can be found in a hybrid communication approach whereby secure transactions, independent from the state of the application, such as authentication and PayPal payments, are carried out via REST API, whereas WebSockets are only used for low-latency interaction and real-time chat synchronization. The implementation of the Modular Monolith and React framework leads to clean code and beautiful UI.

5.2 Future Improvements

With additional development time, the team would prioritize implementing an “Elo-based” matchmaking system and a “Global Leaderboard” to foster a more competitive environment and enhance player retention. We would also develop a "Friend List" as well as a "Direct Invite" system that would allow players to easily invite their friends and start games with them. Finally, to further build a community around the game, we would introduce a "Live Spectator Mode". This mode will enable players to watch live, high-stakes games being played by other people in real time through the use of WebSockets technology with a live reactions (emotes) system as well as spectator chat rooms.

5.3 Architecture Trade-offs & Reconsidered Decisions

Database: MongoDB is suitable for flexible game data, while subscription and payment data would benefit from relational databases such as PostgreSQL for stronger ACID guarantees.

TechStack: Module isolation currently depends on code review practices because JavaScript does not enforce compile-time access restrictions. TypeScript would provide stronger architectural enforcement through static typing.

The current Socket.io implementation operates on a single server instance. Horizontal scaling would require Redis Pub/Sub to synchronize socket events across multiple nodes.

6. References

[1] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021, pp. 3-30.
[2] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002, pp. 152-163 (Repository pattern), 184-196 (Data Mapper pattern).
[3] Meta Open Source, "React - The library for web and native user interfaces," React Documentation, 2024. [Online]. Available: https://react.dev [Accessed: May 2026].
[4] E. You, "Why Vite," Vite Documentation, 2024. [Online]. Available: https://vite.dev/guide/why.html. [Accessed: May 2026].
[5] P. Krisko, "Zustand - Bear necessities for state management in React," GitHub repository, 2024. [Online]. Available: https://github.com/pmndrs/zustand. [Accessed: May 2026].
[6] K. Chodorow, MongoDB: The Definitive Guide, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2019, pp. 1-20.
[7] A. Harris, "ULID Spec - Universally Unique Lexicographically Sortable Identifier," GitHub, 2019. [Online]. Available: https://github.com/ulid/spec [Accessed: May 2026].
[8] PayPal Developer, "Orders v2 API Reference," PayPal Developer Documentation, 2024. [Online]. Available: https://developer.paypal.com/docs/api/orders/v2/ [Accessed: May 2026].
[9] K. Schwaber and J. Sutherland, The Scrum Guide, Scrum.org, 2020. [Online]. Available: https://scrumguides.org/scrum-guide.html [Accessed: May 2026].
[10] Socket.IO, "How it works - Socket.IO Documentation," 2024. [Online]. Available: https://socket.io/docs/v4/how-it-works/ [Accessed: May 2026].
[11] Cloudinary, "Image transformations - Node.js SDK," Cloudinary Documentation, 2024. [Online]. Available: https://cloudinary.com/documentation/node_image_manipulation [Accessed: May 2026].
[12] L. Lovász, "Sharp - High performance Node.js image processing," GitHub repository, 2024. [Online]. Available: https://github.com/lovell/sharp [Accessed: May 2026].
[13] TicTacToang Development Team, "Centralized High-Resolution Architecture and Sequence Diagrams Cloud Repository," [Online]. Available: Assignment 3 Fullstack Development Diagram . [Accessed: May 21, 2026].
[14] TicTacToang Development Team, "TicTacToang API Documentation - Swagger UI," [Online]. Available: https://tictactoang-backend-dt4u.onrender.com/api-docs. [Accessed: May 21, 2026].

AI Usage:

AI tools were used to support specific development tasks while all architectural and implementation decisions remained under team control.

- *Repomix: Serialized the codebase into structured context for AI-assisted development to avoid hallucinations*
- *Stitch: Generated early UI wireframes and mockups for key pages.*
- *Gemini: Assisted with unfamiliar technologies such as Socket.IO, Nodemailer, React toast systems, and PayPal integration.*
- *Claude,Github Copilot: Supported debugging, error tracing, and targeted code review for critical modules.*

All AI-generated output was reviewed, tested, and modified by the team before integration.

7. Contribution Graph

The team's repository is publicly accessible here

The table below maps each GitHub username to its corresponding team member's full name, as several usernames do not contain the member's full legal name. The contribution metrics is here

Full Name	Role	Assigned Tasks	Score
Nguyen Q. Khanh (KhanhQNguyn)	Project Lead, Frontend Developer	Sheet for report tracking, report writing and refinement, UI/UX design configuration (sound, notification toast, theme, music, font, color), game board implementation, marker renderer, online room connection, disconnect grace period, match replay, premium subscription, component replenish	20%
Hoang M. Thang (ThangHoang54)	Backend Lead	Led the backend architecture design and implemented the N-Tier module structure. Developed the Authentication module with JWT middleware, Profile module, Admin module and managed the Room and WebSocket (Socket.IO) lifecycle. Designed the database schema and MongoDB modeling, implemented integration testing, created seed data scripts, reformat code and co-developed 22 RESTful API endpoints.	20%
Nguyen D. G. Phat (giaphat060206)	Frontend Developer	Frontend UI Implementation for the following pages: Guest & Admin's Pages; Player's Select Game Mode, Lobby, Customize, Profile. Easy AI Logic for Singleplayer. Dynamic Mobile UI implementation	20%
Tran H. Minh (Minz516)	Scrum Master, QA Engineer	Sprint planning and task management, GitHub project board maintenance, system-wide quality assurance, performance improvement, test case design and execution, meeting facilitation and retrospectives	20%
Truong K. Minh (kiemminh000, Minh-bot73)	Backend Developer	Co-designed the backend architecture and engineered technical documentation, including backend component diagrams and database schema optimization. Spearheaded the Subscription module (featuring PayPal payment gateway and automated email API integration) and the core Game module logic. Co-developed the Authentication, Room management systems, real-time WebSocket (Socket.IO) event handling, and co-developed others 22 RESTful API endpoints.	20%

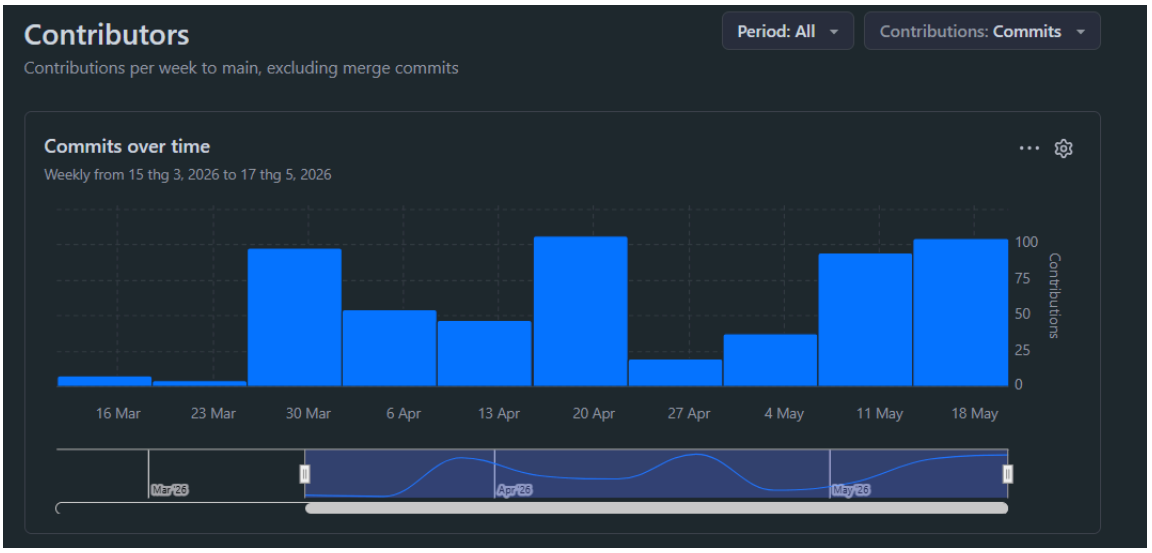

Figure 9: Contribution Graph

8. Appendix

8.1 Desktop & Mobile UI

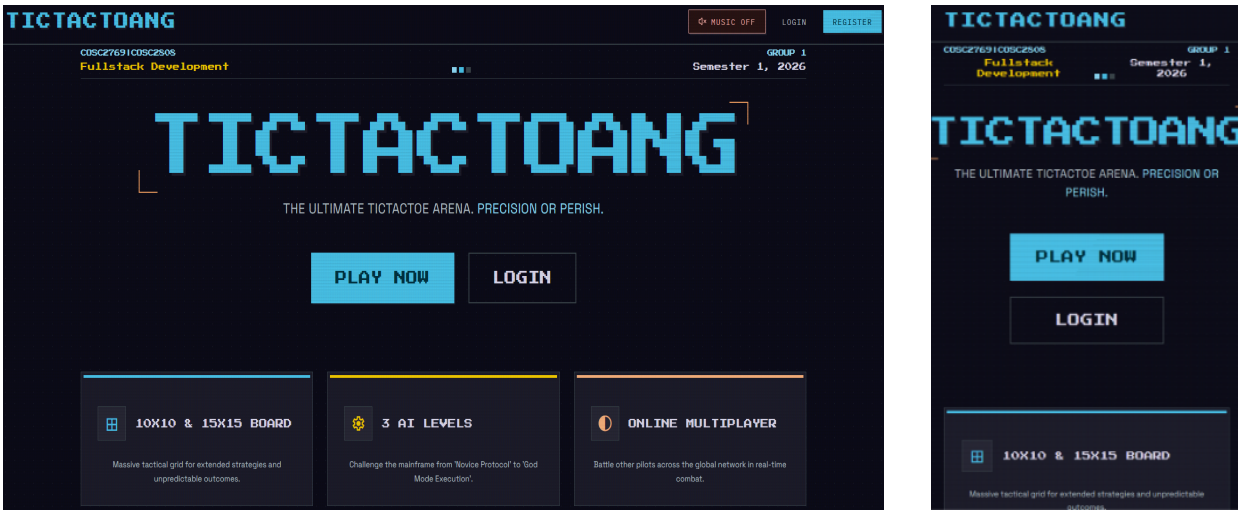

Figure 10: TicTacToang Landing Page - Desktop & Mobile UI

Figure 11: TicTacToang Room Customize - Desktop & Mobile UI

8.2 Detailed Contribution Score

Table 6: Detailed Commit Metrics

Full Name	GitHub Username	Total Commits	Lines Added	Lines Removed
Hoang M. Thang	ThangHoang54	144	27,414	11,943
Nguyen D. G. Phat	giaphat060206	100	21,183	8,998
Nguyen Q. Khanh	KhanhQNguyn	88	24,885	21,339
Tran H. Minh	Minz516	45	9,780	5,119
Truong K. Minh	kiemminh000 / Minh-bot73	75 + 81 = 156	4.826	2764